Hochschule Fulda
University of Applied Sciences

Master Team Project Fall 2025

Team-2

FuldaNexus

Name	Email	Back end or front,End affinity/lead or GitHub master
Peddada Ramachandra Sai Darshan	ramachandra-sai-darshan.peddada@informatik.hs-fulda.de	Team Lead, QA and document master
Tanmay Manjunath	tanmay.manjunath@informatik.hs-fulda.de	Front End Lead
Sahaj Bikram Malla	sahaj bikram.malla@informatik.hs-fulda.de	Back End Lead, DB
Bikrant Bidari	bikrant.bidari@informatik.hs-fulda.de	GIT Master and Cloud
Meet Kalathiya	meet-mukeshbhai.kalathiya@informatik.hs-fulda.de	Frontend
Karan Kamath	karan-karunakar.kamath@informatik.hs-fulda.de	Backend

Milestone - 2

History Revision Table:

Version	Revised on
1.0	19.11.2025

INDEX

SR NO	TITLE	PAGE
1	Functional Requirements	3
2	List of Main data items	5
3	UI Mocks and Storyboards	7
4	High-Level Architecture	8
5	High-Level UML Diagrams	13
6	Key Risks	16
7	Project Management	17

1. FUNCTIONAL REQUIREMENTS

1.1 Common Features

- **Register and User Authentication-(P1)**
  The system allows university users to register with a verified campus email and log in securely. Guest access is allowed for limited browsing.
- **User Roles-(P1)**
  The system shall support distinct roles (Attendee, Event Organizer, Administrator) with specific permissions.
- **Profile Management-(P1)**
  Users shall be able to view and edit profile details (name, contact, interests).
- **Password Management-(P1)**
  Users shall be able to reset their passwords.

1.2 Event Creation & Management (Organizer)

- **Create Events-(P1)**
  Authorized Organizers shall be able to create event listings with title, description, date, time, location (physical/virtual), capacity, and category.
- **Ticket Management-(P1)**
  Organizers shall specify ticket types (free, paid, VIP), prices, quantities, and availability.
- **Media Upload-(P1)**
  Organizers shall be able to upload event-related images and videos.
- **Edit/Cancel:** Organizers shall be able to view, edit, or cancel their own events.
- **Attendee Management-(P1)**
  Organizers shall manage attendee lists and check in attendees.
- **Analytics Dashboard-(P4)**
  Organizers shall have a dashboard to view registration numbers, attendance, and ticket sales.

1.3 Event Discovery & Search

- **Event Listing-(P1)**
  The system shall display a comprehensive list of upcoming and past events.
- **Basic Search-(P1)**
  Users shall be able to search by keywords, categories, dates, and locations.
- **Event Details-(P1)**
  The system shall display details including schedule, location map, and organizer contact.
- **Advanced Filtering-(P2)**
  Users shall be able to filter by "free events," departments, or event type.
- **Semantic Search -(P4)**
  The system shall support natural language queries (e.g. "workshops for public speaking") and provide recommendations based on semantic understanding and user interests.
- **Favourites-(P3)**
  Users shall be able to add events to a personal "watch" list.

1.4 Registration & Ticketing

- **Registration-(P1)**
  Users shall be able to register for free events or purchase tickets for paid events.
- **Ticket Generation-(P1)**
  The system shall generate unique digital tickets/QR codes for successful registrations.

- **My Tickets-(P1)**
  Users shall view a record of their registered/purchased tickets in their profile.

1.5 Event Safety SOS Co-pilot (USP)

- **SOS Button-(P1)**
  The system shall provide an easily accessible panic button within the event Chat interface.
- **Security Alert-(P1)**
  Activation shall immediately alert Event organizer with the user's ticket details and event context.
- **Incident Logging-(P4)**
  Security/Admins shall be able to log incident details linked to specific events.
- **Real-time Comms-(P4)**
  The system shall provide a communication channel between the user and security personnel.

1.6 Rewards

- **Track Engagement-(P1)**
  The system shall track actions like attendance and interactions.
- **Points & Badges-(P2)**
  Users shall earn points and badges based on predefined criteria.
- **Profile Display-(P2)**
  Earned badges and points shall be visible on the user profile.
- **Leaderboard-(P2)**
  A leaderboard shall display top-ranking users.
- **Reward Redemption-(P1)**
  Users shall be able to redeem points for rewards (e.g. discounts. Coupons) .

1.7 Ticket Reselling

- **List for Resale-(P1)**
  Users shall be able to list purchased tickets for resale.
- **Secure Transfer-(P1)**
  The system shall facilitate the secure transfer of ticket ownership from seller to buyer.
- **Resale Marketplace-(P1)**
  A dedicated interface for browsing resale tickets.

1.8 Community Chat

- **Event Chat Rooms-(P1)**
  A dedicated chat room shall be created for each event, accessible only to registered attendees.
- **Messaging-(P1)**
  Users shall be able to send text messages within the chat.
- **Moderation-(P1)**
  The system shall support reporting of inappropriate content in chats.

2 List of main data items and entities

2.1 User & Profile Data

- **Meaning:** Stores account and personal details of a registered user.
- **Fields:** UserID, FullName, Email, Password (hashed), Role (User/Administrator), ProfilePictureURL, Bio, Status, JoinedDate.
- **Usage:** Represents individual user identity and manages access levels and personalization

2.2 Event

- **Meaning:** Represents a specific activity organized within or near Fulda (e.g., academic workshop, cultural event, sports meetup).
- **Fields:** EventID, Title, Description, OrganizerID, CategoryID, Date, StartTime, EndTime, Location, MediaURLs, ApprovalStatus, CreatedAt, UpdatedAt.
- **Usage:** Central object connecting users, participation records, and chats. Viewable through event listings and detail pages.

2.3 Event Participation

- **Meaning:** Represents a user's registration and participation status in an event.
- **Fields:** ParticipationID, EventID, UserID, RegistrationDate, Status (Registered/Cancelled/Attended).
- **Usage:** Tracks attendance and grants access to the event's chat and notifications

2.4 Category

- **Meaning:** Defines types of events to improve discoverability and organization.
- **Fields:** CategoryID, Name, Description.
- **Usage:** Enables users to browse or filter events based on themes such as Sports, Academic, Cultural, Volunteer, etc.

2.5 Chat

- **Meaning:** Represents the real-time discussion area associated with an approved event.
- **Fields:** ChatID, EventID, CreatedAt.
- **Usage:** Automatically created when an event is approved. Only accessible to registered participants of that event.

2.6 Message

- **Meaning:** Individual communication sent within an event chat.
- **Fields:** MessageID, ChatID, SenderID, Content, Timestamp, IsEdited, IsDeleted.
- **Usage:** Enables communication between participants and organizers for coordination or questions.

2.7 Media

- **Meaning:** Represents media items (images, flyers, videos) uploaded for events or user profiles.
- **Fields:** MediaID, UploadedByUserID, AssociatedEventID, FileURL, MediaType (Image/Video/Link), UploadedAt.
- **Usage:** Enhances event descriptions and user profiles with visual content.

2.8 Notification

- **Meaning:** Alerts users about important actions related to events or participation.
- **Fields:** NotificationID, RecipientID, Type (System/Event/Chat), Message, Status (Read/Unread), Timestamp.
- **Usage:** Keeps users informed about event approvals, new messages, or updates.

2.9 Report

- **Meaning:** Used by users to report inappropriate content or activities.
- **Fields:** ReportID, ReporterID, TargetType (User/Event/Message), Reason, Timestamp, Status.
- **Usage:** Sent to administrators for moderation and action.

2.10 AuditLog

- **Meaning:** Records all administrative actions and moderation activities for accountability.
- **Fields:** LogID, AdminID, ActionType, TargetEntityID, EntityType, Timestamp, Notes.
- **Usage:** Provides traceability and ensures compliance with moderation policies.

2.11Feedback (Optional Extension)

- **Meaning:** Allows users to give feedback about attended events.
- **Fields:** FeedbackID, EventID, UserID, Rating, Comment, Timestamp.
- **Usage:** Builds trust and helps organizers improve event quality.

2.12 Rewards

- **Meaning:** Tracks user engagement and rewards through credits earned for platform activity.
- **Fields:** CreditID, UserID, TotalCredits, EarnedFrom (ActivityType), RedeemedCredits, RedemptionDate, Level (Bronze/Silver/Gold).
- **Usage:** Encourages consistent user engagement; credits can be redeemed for discounts on paid event tickets.
- **Rules:**
    - Credits are earned by joining or creating events, or engaging in chats.
    - Credits cannot be exchanged for money.
    - Credit balance and level are shown in the user's profile.

2.13Ticket & Reselling

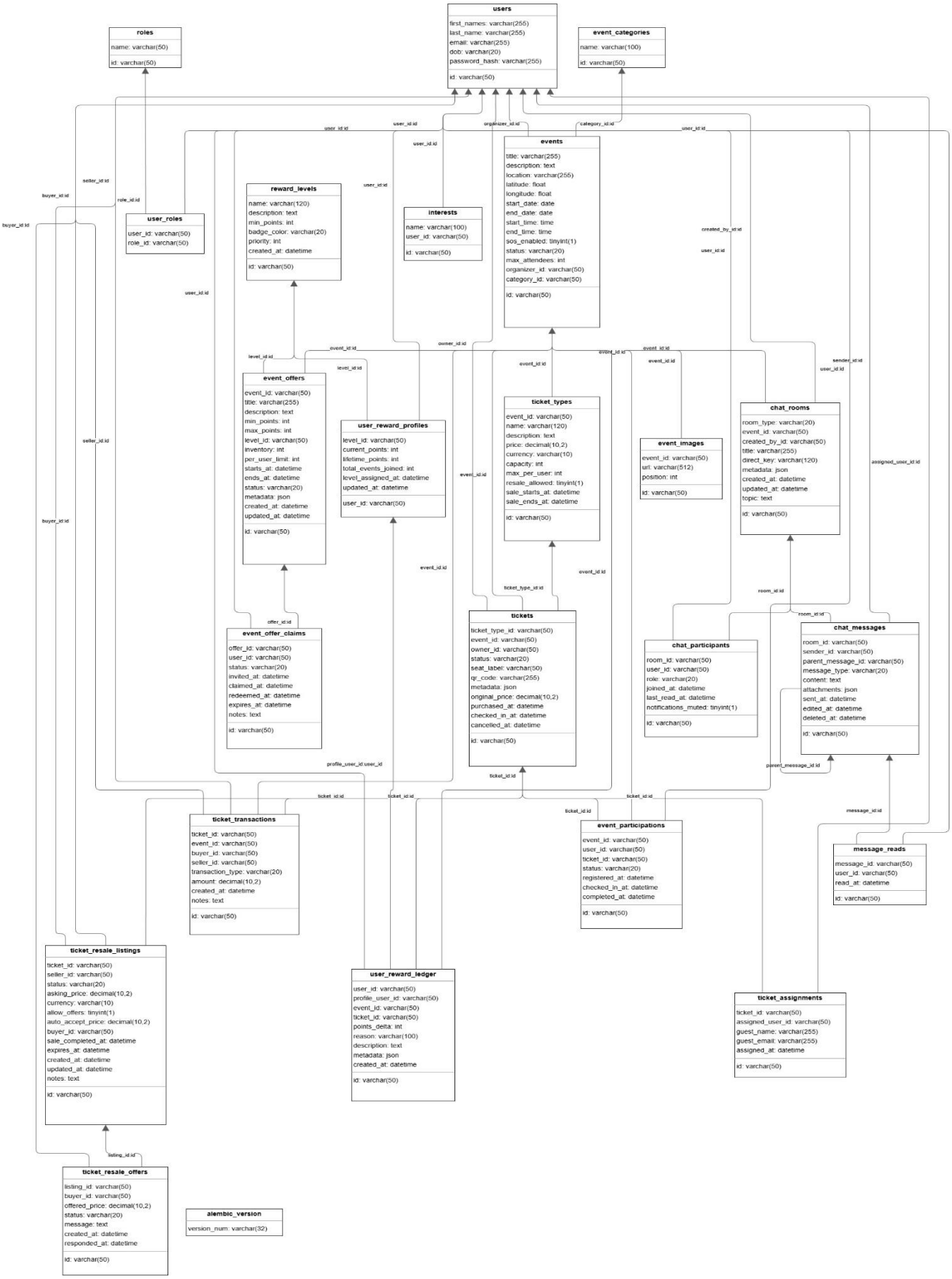
- **Meaning:** Represents tickets purchased by users for specific events, including resale capabilities.
- **Fields:** TicketID, EventID, BuyerID, PurchaseDate, Price, Status (Active/Cancelled/Resold/Used), IsRefundable, ResellEligible, ResoldToUserID, OriginalPrice.
- **Usage:** Tracks ticket ownership, supports reselling, and ensures compliance with resale rules.
- **Rules:**
    - A user may purchase **up to 4 tickets per event**.
    - Events marked **non-refundable** may allow **reselling** if the organizer permits.
    - Reselling is only allowed **on the platform** and **at or below the original purchase price (MRP)**.
    - Organizers decide at event creation whether **reselling** is permitted.
    - Tickets cannot be resold after being marked as "Used" or after the event date.

3 UI Mock-ups and Storyboards

The UI mock-ups and the storyboards as per use cases can be found on the miro board link access: https://miro.com/app/board/uXjVJslq-KQ=/

4 New High-level Architecture, Database Organization

4.1 Database Organization



DB Organisation: The application's database is organized around core entities such as Users, Events, Tickets, and Chat Rooms, with clear relationships between event organizers, attendees, and ticket resellers. It follows a relational schema optimized for event management and marketplace dynamics, supporting complex features like ticket resale transactions, gamified user rewards, and real-time messaging while ensuring secure data handling through role-based access control.

4.2 Storage

we decided against using database BLOBs. Instead, all large binary objects, which include **images and videos** (such as event banners and profile pictures), will be stored externally in **AWS S3**. This strategy is crucial for maintaining high application performance and scalability, as it prevents the MySQL database engine from slowing down by having to handle large files. All uploaded media will be in standard web formats like PNG/JPEG. The database will only store the **URL links** pointing to the files in the S3 storage bucket.

4.3 Search/filter architecture and implementation

The search system is engineered for both speed and intelligence, supporting fast keyword matching and sophisticated queries. The foundational method for **Standard Keyword Search** uses optimized **SQL commands** where the backend (FastAPI) executes pattern matching using the **%LIKE% operator** against key columns like Title and Description in the Events table. To maintain high performance, these target fields are **indexed in MySQL** to prevent slow full-table scans. For general filtering, the system uses precise **SQL WHERE clauses** to allow users to narrow results by CategoryID, date range, or Price(e.g. searching for free events).

The unique **Advanced / Semantic Search** feature requires a more complex processing layer to interpret natural language queries **without using an LLM** . Instead of simple keyword matching, the system will employ **Natural Language Processing (NLP)** and **advanced algorithms** to interpret the user's intent when they type a phrase like "workshops for aspiring entrepreneurs". This processing layer translates the natural language query into structured search criteria (keywords, categories, intent, and synonyms). The system then executes a refined query against the database, prioritizing events that semantically match the user's interests for a more relevant and organized result set.

4.4 Optimization

- **Database Speed Boost (Indexing):** We put **Database Indexes** on the columns used for searching, like event titles and descriptions. This is like adding an alphabetized index to a book: the database avoids reading every single record, making the basic keyword search much faster.
- **Smart Filtering:** The website gives users many ways to narrow their search results (eg, picking a date range or category). We use these selections to filter the data right in the database query. This ensures the computer only processes a small, relevant set of events, reducing the time needed to find the final answer.
- **Relevance Ranking:** Results aren't just listed by date. We use a **Relevance Score** to decide what the user sees first. This score is calculated based on how well the event matches the user's intended query and what we know about their past interests. This ensures the most helpful events appear at the top.
- **Term Expansion (NLP):** When a user searches, our system uses **Natural Language Processing (NLP)** to automatically expand their terms. For example, if they search for "hacks," the

system might also look for "hackathon" or "coding competition." This ensures users find relevant events even if they don't use the organizer's exact wording.

- **Efficient Sorting:** When a user chooses a specific way to sort the results (like by price or event date), we apply that sorting rule directly in the initial database command. This is faster than retrieving all the data first and then sorting it later in the application.

4.5 API Outline

Public Event Discovery (no auth)

1. GET /api/events

- List upcoming/past events with pagination + filters.

- Query: status, categoryId, keyword, fromDate, toDate, location.

- Response: 200 OK [EventSummary[]].

2. GET /api/events/{eventId}

- Full event details (schedule, organizer contact, ticket tiers, media).

- Response: 200 OK [EventDetail].

Authentication (login required beyond this point)

1. POST /api/auth/register – create attendee/organizer profile.

2. POST /api/auth/login – issue JWT + refresh token.

3. POST /api/auth/logout – revoke refresh token.

4. POST /api/auth/forgot-password / POST /api/auth/reset-password.

User Profiles

1. GET /api/users/me – profile + roles + reward stats.

2. PUT /api/users/me – update name, phone, interests, avatar.

3. GET /api/users/me/tickets – “My Tickets” list with QR status.

4. GET /api/users/me/events – List all user-created events.

5. GET /api/users/me/tickets – List all tickets resold / yet to be resold.

Organizer Event Management

2. POST /api/events – create event (title, schedule, venue coords, capacity, category, SOS flag, virtual link).

3. PUT /api/events/{eventId} – update organizer-owned event draft.

4. DELETE /api/events/{eventId} – cancel event (notifies registrants).

Registration & Ticketing

1. POST /api/events/{eventId}/registrations – register / purchase ticket (handles payment token when needed).

2. GET /api/registrations/{registrationId} – view registration.

3. POST /api/tickets/{ticketId}/check-in – organizer/staff check-in via QR payload.

4. GET /api/tickets/{ticketId}/qr – fetch signed QR data.

Ticket Resale

1. POST /api/tickets/{ticketId}/resale-listings – list eligible ticket for resale.

2. GET /api/resale/listings – browse marketplace (auth required).

3. POST /api/resale/listings/{listingId}/offers – place offer/buy.

4. PATCH /api/resale/listings/{listingId} – seller updates price/status.

5. PATCH /api/resale/offers/{offerId} – accept/decline/withdraw.

Rewards & Engagement

1. GET /api/rewards/profile – points, badges, tier.

2. GET /api/rewards/leaderboard – top users.

3. POST /api/rewards/redemptions – redeem reward offer.

4. GET /api/rewards/redemptions – redemption history.

SOS Safety

1. POST /api/events/{eventId}/sos – attendee triggers SOS with location + optional note.

2. GET /api/security/incidents (admin only) – list active SOS incidents.

3. PATCH /api/security/incidents/{incidentId} (admin/security) – acknowledge/resolve, add notes.

Community Chat

1. GET /api/events/{eventId}/chat/room – fetch room info (registered attendees only).

2. GET /api/chat/rooms/{roomId}/messages – paginated history.

3. POST /api/chat/rooms/{roomId}/messages – send text/attachment.

4. POST /api/chat/messages/{messageId}/reports – attendees flag content.

5. GET /api/chat/rooms/{roomId}/moderation (admin) – review flagged chat content.

6. PATCH /api/chat/messages/{messageId} (admin) – hide/delete violating message.

Admin & Moderation

1. GET /api/admin/events/pending – list events awaiting approval.

2. PATCH /api/admin/events/{eventId} – approve/reject event (body { status, reviewerNotes }).

3. POST /api/admin/events/{eventId}/deactivate – remove published event (soft delete with reason).

4. GET /api/admin/event-modifications – review organizer change requests.

5. PATCH /api/admin/event-modifications/{requestId} – approve/reject modifications.

6. GET /api/admin/users – search/filter users.

7. PATCH /api/admin/users/{userId}/status – activate/deactivate accounts.

8. GET /api/admin/sos-incidents – alias of security incidents for dashboards.

9. GET /api/admin/content-reports – aggregated chat/event-content flags.

10. PATCH /api/admin/content-reports/{reportId} – resolve report, optionally remove offending content.

Access Control Notes

- GET /api/events and GET /api/events/{id} remain public.

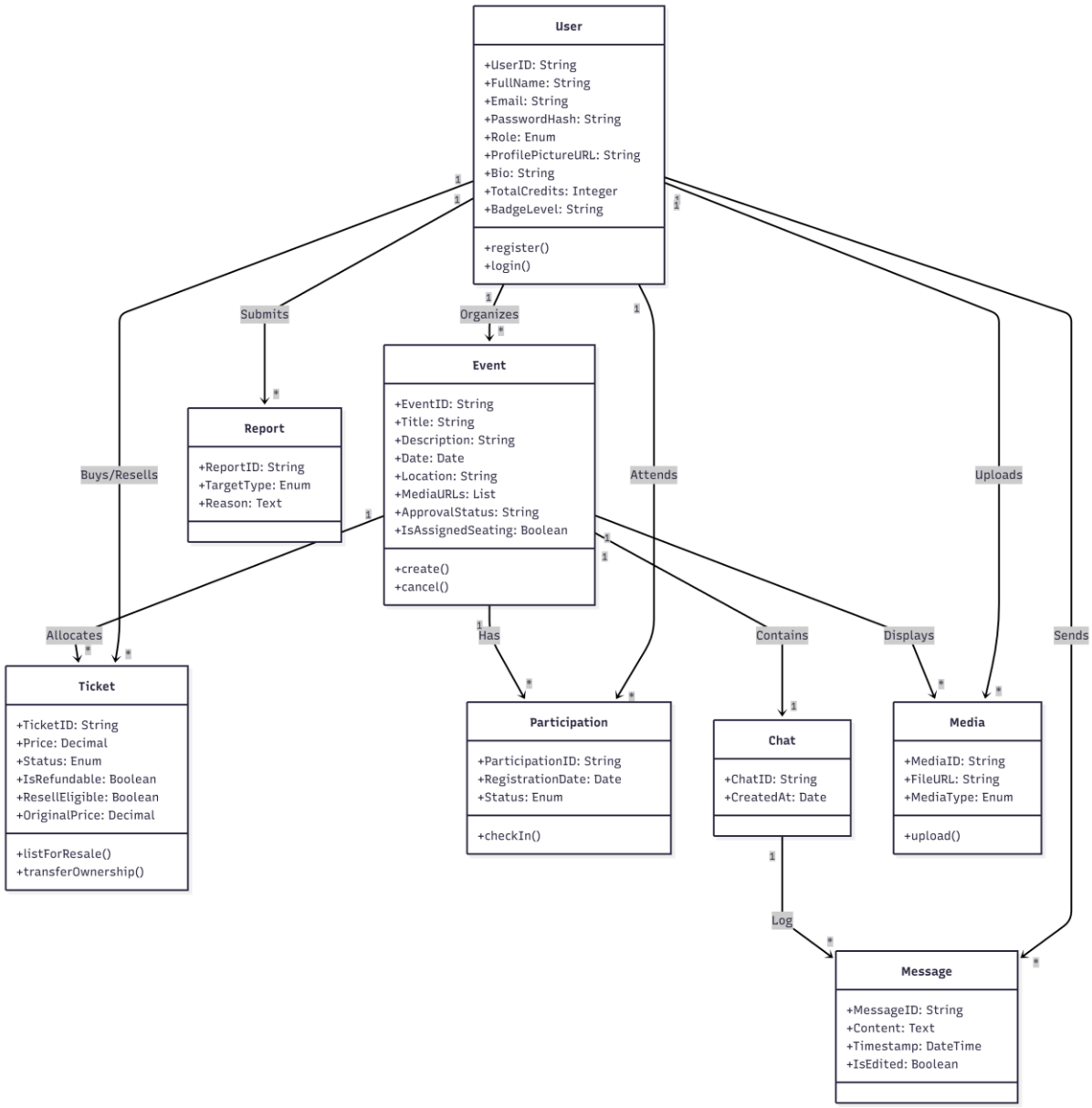
- All other endpoints require authenticated JWT.

- Organizer-only routes enforce role checks; admin endpoints restricted to Administrator role.

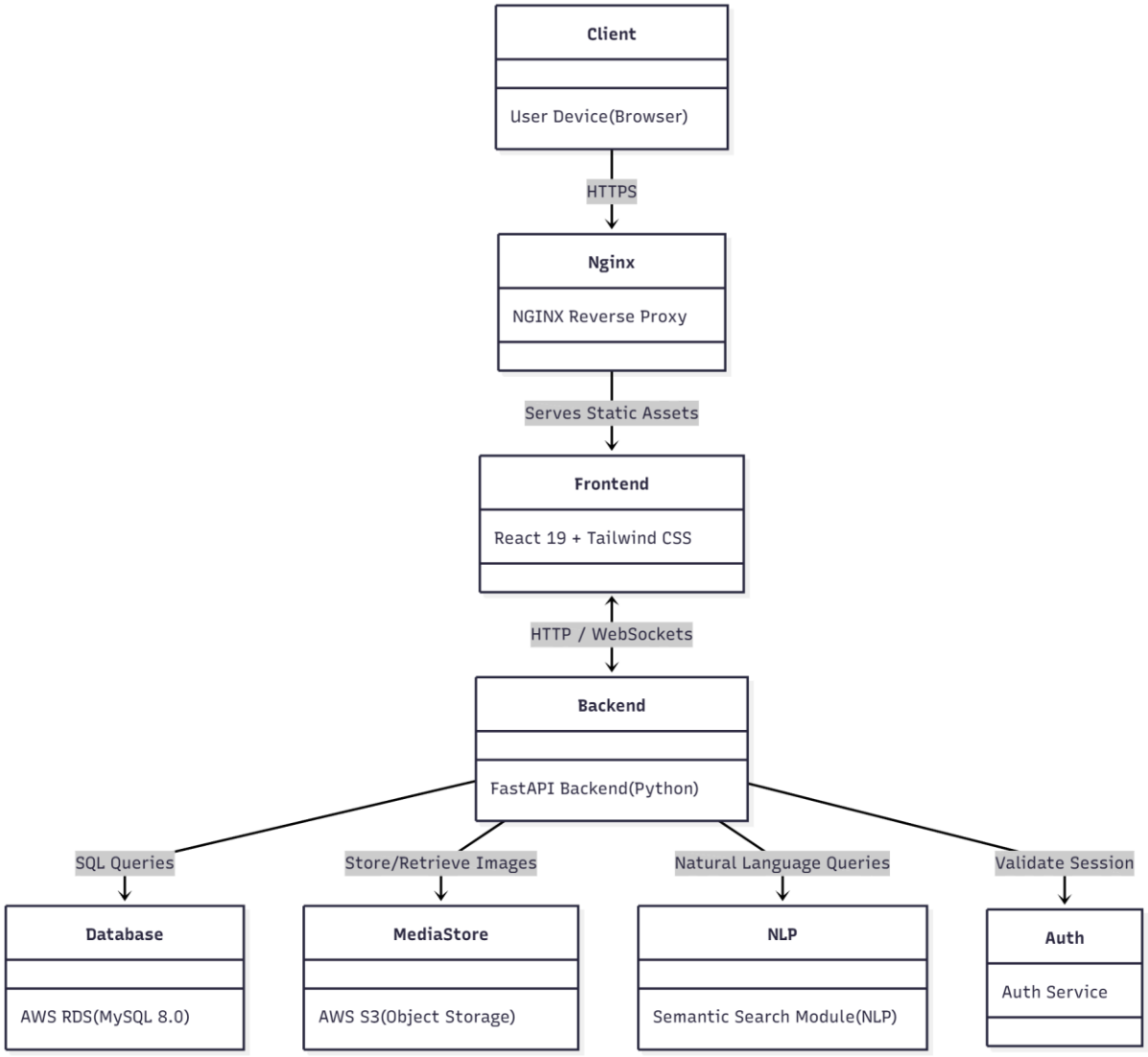
- SOS endpoints: creation requires attendee auth; viewing/resolving limited to Admin/Security roles.

5 HIGH-LEVEL UML DIAGRAMS

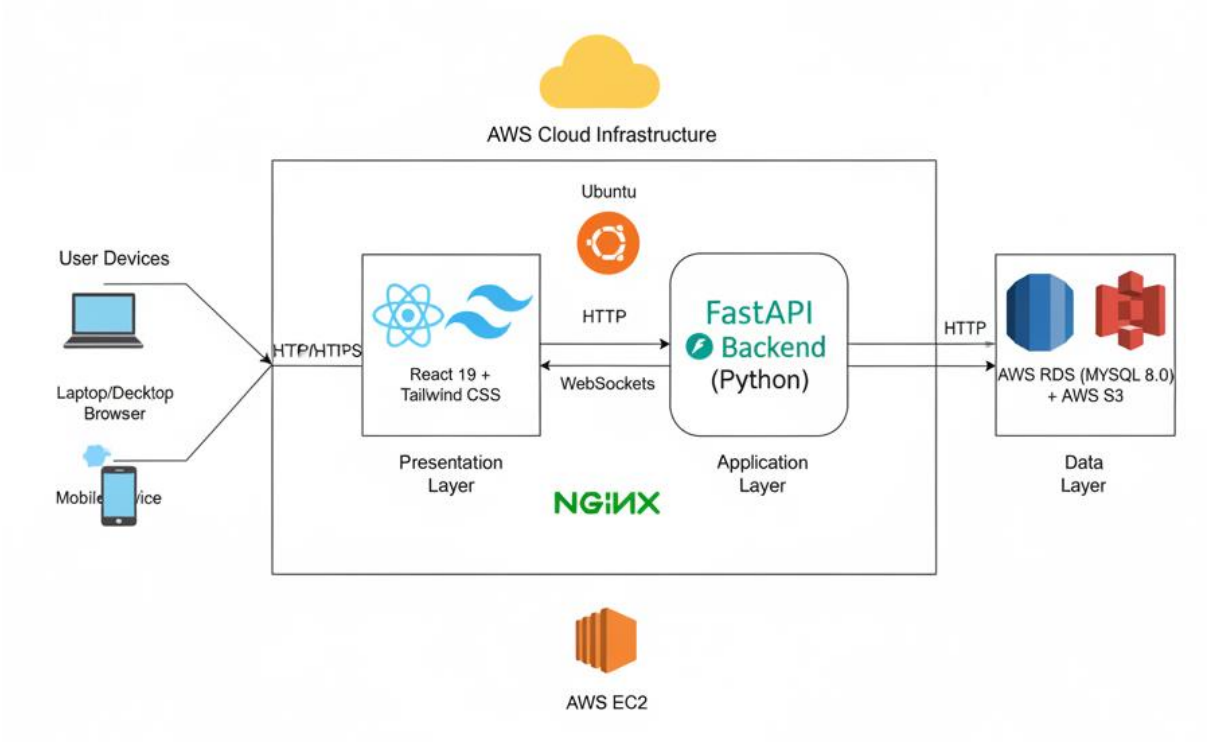
5.1 Class Diagram



5.2 Component Diagram



5.3 Deployment Diagram



6 Key Risks

6.1 Skill Risks

Risk: The team has selected **React 19** and **FastAPI** , but the checklist indicates that some members still "need to learn" and are currently practicing these frameworks. There is a risk that the learning curve for these specific technologies will slow down the initial development phase.

Mitigation: The team has already started to learn the technologies and are trying to learn most of it before setting up the project, and we also track the progress of each member.

6.2 Schedule Risks

Risk: The project commits to implementing four distinct USPs (SOS Co-pilot, Rewards, Semantic Search, and Secure Reselling) in addition to standard event management. Developing complex features like a "secure in-app marketplace" for tickets and a “semantic search engine “within the semester timeline poses a significant risk of missing deadlines or delivering incomplete features**.**

Mitigation: We will strictly prioritize the "SOS Co-pilot" and basic "Event Management" (P1 features) for the initial release. Complex features like "Semantic Search" and "Reselling" will be treated as lower priority (P2/P3) and simplified if the schedule slips.

6.3 Technical Risks

Risk: The **SOS Co-pilot** now relies on **static ticket data** (Seat Number, Row) to locate a user during an emergency. A major technical risk is handling events with **"Free Seating"** or **"Standing"** areas where ticket data does not correspond to a physical location, potentially sending security to the wrong place.

Mitigation: We will implement a database flag for “is_assigned_seating”. For non-assigned events, the SOS interface will prompt the user to select a "Zone" (eg, "Near Stage", "Entrance") manually before sending the alert.

6.4 Teamwork Risks

Risk: “Reduced Backend Capacity” We are currently down one backend developer, significantly increasing the workload on the remaining backend lead. This creates a critical bottleneck for delivering the APIs required for the Frontend team and implementing complex logic for the Reward Engine and SOS features.

Mitigation: We will implement **Cross-Functional Support**, where Frontend members with Python knowledge will assist with simpler CRUD API endpoints. We will also simplify the backend logic for "Reward" and “Systematic Search” to ensure the remaining backend resources can focus on the critical "SOS" and "Auth" services.

6.5 Legal/Content Risks

Risk: The application requires "media-rich" content, allowing organizers to upload images and videos for events. There is a risk of users uploading copyrighted material without permission, creating liability for the platform

Mitigation: We will include a mandatory "Terms of Service" checkbox confirming copyright ownership during upload and implement a "Report Content" feature for the community to flag violations.

7 Project Management

We have shifted from the M1, into the hands-on building phase, which is M2. Our main aim right now focuses on getting things done efficiently, with different teams handling their parts at the same time. To push that efficiency further, we picked up a way of working that keeps things separate, along with something called the Contract-First method. In this setup, the people working on the front end and those on the back end can mostly go their own ways. They just follow some straightforward guidelines to stay in line with each other. We nailed down every detail of the API setups using OpenAPI and Swagger tools, all before jumping into any major coding work. That acts like the main guide for how data moves between the sides. Thanks to it, the front-end group, which uses React 19, can start creating and checking out their user interface pieces right away. They use fake data for that, so they do not wait around for the back-end folks, who rely on FastAPI, to sort out the database stuff. On top of that, since the back-end side has less manpower these days, we brought in a system of cross-team help. It works by giving some skilled front-end workers the easier back-end jobs, such as tweaking fields in the database. This helps even out the overall workload across the project.

For task tracking, we rely on **GitHub Projects**, which functions as our central, unified dashboard. This system ensures total transparency for all activities. Our board tracks tasks through defined status columns: *Backlog*, *In Progress*, *In Review (QA)*, and *done*. Crucially, every task card is assigned to a single **"Assignee"** (Person in Charge) and given a fixed **"Deadline"**. Cards lacking these two fields are blocked from starting development. For code integrity, **no code is merged** into the main master branch without a formal **Pull Request review** by the Git Master or the respective domain lead. We maintain communication via **WhatsApp** for urgent issues and pair programming. We hold mandatory **weekly offline syncs (**Extra Offline if required) to review the board, update deadlines, and resolve any interface issues between the frontend and backend.